CS-107 Lecture 21
Scripting, Permission, Wildcards and Process Control

Troy Astarte

Megan Venn-Wycherley

Goals for Today

- Variables and arrays in bash: assignment, quoting and indexing
- Functions: defining them and returning values
- Writing and running scripts: nano, the shebang and file permissions
- Wildcards: matching filenames with * and ?
- Process control and editing keys: CTRL+C, CTRL+Z, fg, CTRL+K/U/Y

Variables

Variables

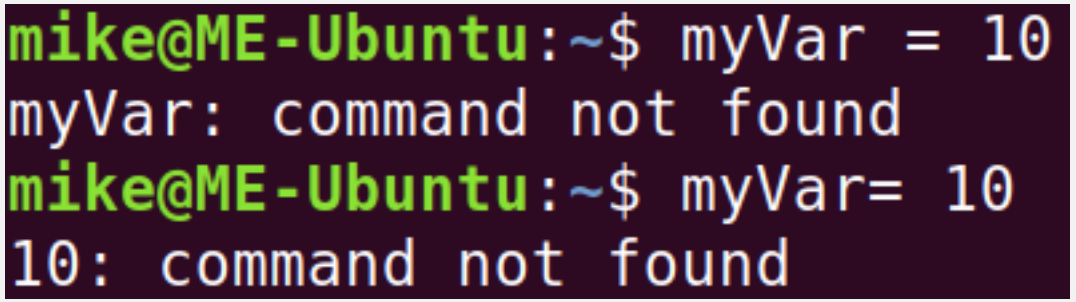
- Declared when needed and **typeless by default**
- In the form: `myVar=value`
- Assigned from **right to left**
- Note the lack of spaces! Why?

Variables

- Issues with spaces:

e.g.:

```
>> myVar = 10
myVar: command not found
>> myVar= 10
10: command not found
```

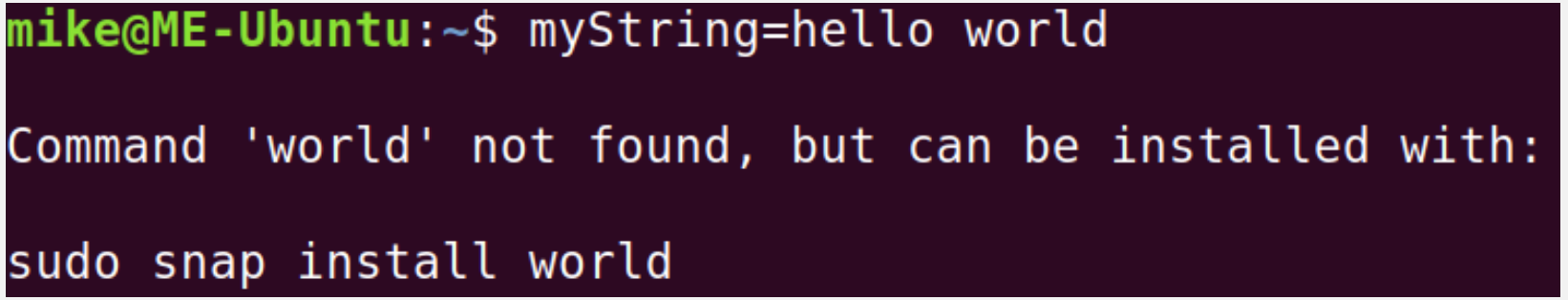


Variables

- Issues with spaces **within values**:

e.g.:

```
>> myString=hello world
world: command not found
>> myString=”hello world”
```



Variables

- Correct without the spaces:

e.g.:

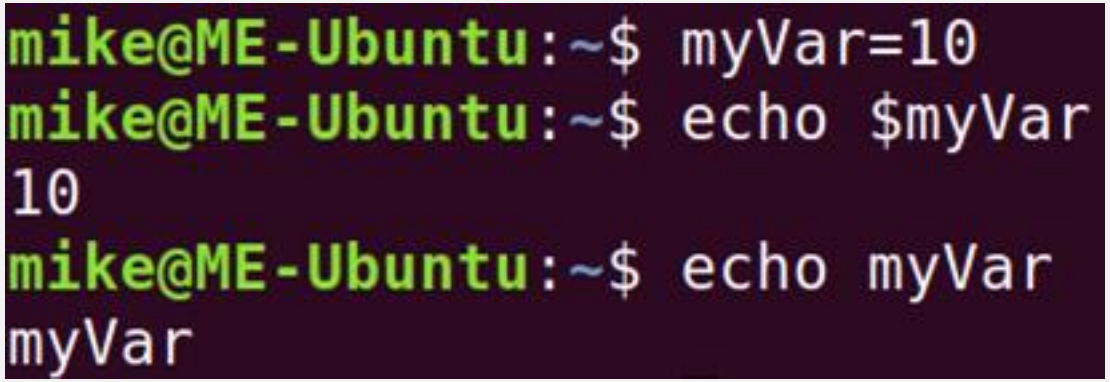
```
>> myVar=10
>> myString=”Hello World”
```

Variables

- Reference a variable with $varname

e.g.:

```
>> myVar=10
>> echo $myVar
10
>> echo myVar
myVar
```



Variables: Numbers

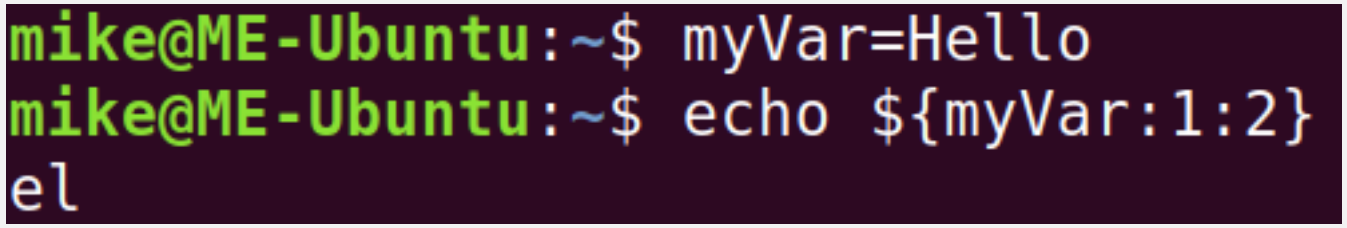
- BASH can interpret variables as different types when needed.
- You can store a float (e.g. 3.14) as a string, but then use it as a float when required.
- More on this in the arithmetic section.

Variables: Strings

- BASH stores its variables as Strings
- Index using the notation:
  - ${String:position:length}

e.g.:

```
>> myVar=Hello
>> ${myVar:1:2}
el
```



Arrays

Arrays

- You can also declare an array as follows:
  - array=(value value value)
- Index with ${array[index]} (from 0)

e.g.:

```
>> A=(cat dog fish)
>> echo ${A[1]}
dog
```

Conditionals

Conditionals: if

- Form:

  if [condition]

  then

    statements

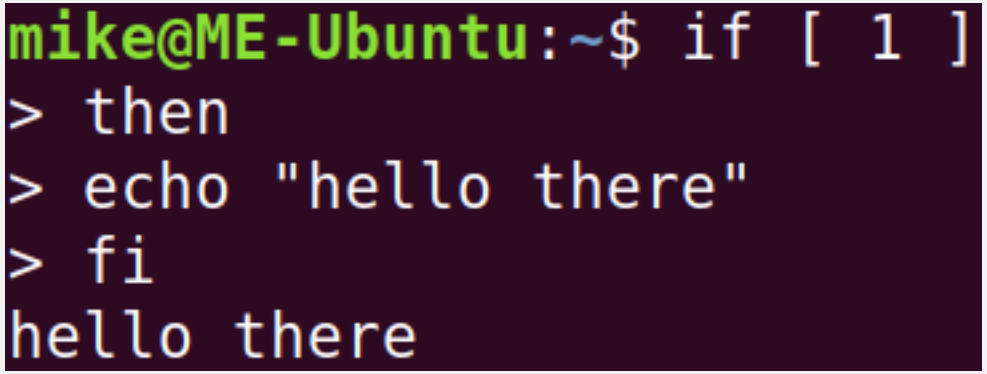
      fi

Note the spaces in the condition

Conditionals: if

e.g.:

```
>> if [ 1 ]
> then
> echo “hello there”
> fi
hello there
```



Conditionals: if elif else

- Form:

  if [condition]

  then

  Statements

  elif [condition]

  Statements

  else

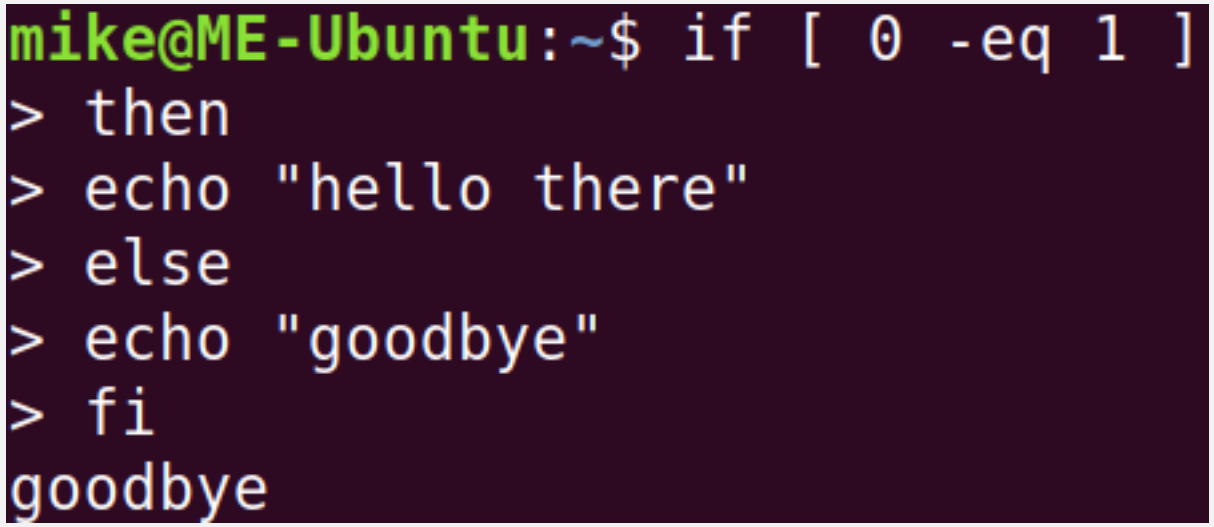
  Statements

  fi

Conditionals: if elif else

e.g.:

```
>> if [ 0 -eq 1 ]
> then
> echo “hello there”
> else
> echo “goodbye”
> fi
goodbye
```



Comparisons

String Comparison expressions

- Equal to: ==
- Not Equal: !=
- Less than / Greater than: < / >
- Empty / Not Empty string: -z / -n

String Comparison expressions

e.g.:

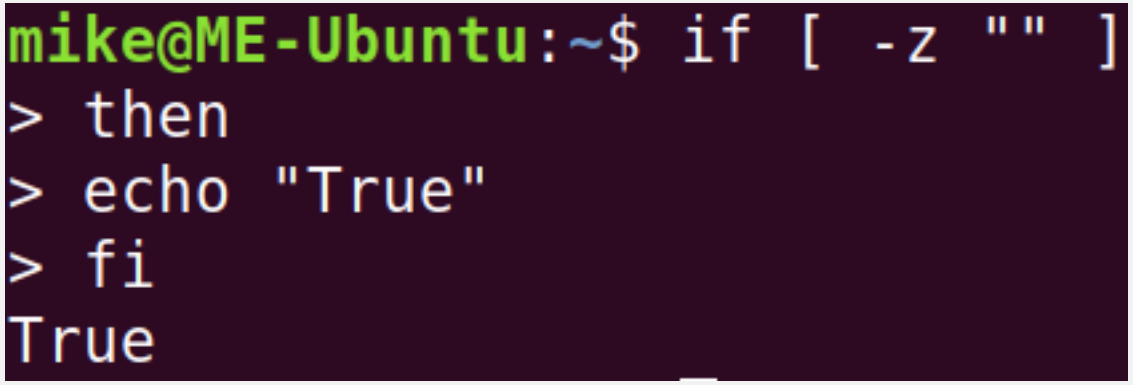
```
“cat” == “cat” # True
-z “” # True
-z “cat” # False
```

```
mike@ME-Ubuntu:~$ if [ "cat" == "cat" ]
then
echo "True"
fi
True
```

String Comparison expressions

e.g.:

```
“cat” == “cat” # True
-z “” # True
-z “cat” # False
```



String Comparison expressions

e.g.:

```
“cat” == “cat” # True
-z “” # True
-z “cat” # False
```

```
mike@ME-Ubuntu:~$ if [ -z "cat" ]
> then
> echo "True"
> fi
```

Numerical Comparison expressions

- Equal to: -eq
- Not Equal: -ne
- Less than / Greater than: -lt / -gt
- lt or Equal / gt or Equal: -le / -ge

Numerical Comparison expressions

e.g.:

```
1 -eq 1 # True
1 -lt 1 # False
1 -le 1 # True
```

```
mike@ME-Ubuntu:~$ if [ 1 -eq 1 ]
> then
> echo "True"
> fi
True
```

Numerical Comparison expressions

e.g.:

```
1 -eq 1 # True
1 -lt 1 # False
1 -le 1 # True
```

```
mike@ME-Ubuntu:~$ if [ 1 -lt 1 ]
> then
> echo "True"
> fi
```

Loops

Loops: for

- Form:

for iterator

do

statements

done

- Iterator can be different type, depending on task

Loops: for

- Can be a list of values:

for element **in** value value value

e.g.:

```
>> for word in hello world
> do
>echo “The word is $word”
> done
The word is hello
The word is world
```

Loops: for

```
mike@ME-Ubuntu:~$ for word in hello world
> do
> echo "The word is $word"
> done
The word is hello
The word is world
```

Loops: for

- Can be a "traditional" style iterator:

for ((init; condition; inc/dec)) #Note spaces

e.g.:

```
>> for (( i = 0; i < 2; i++ ))
> do
>echo "The number is $i"
> done
The number is 0
The number is 1
```

Loops: for

```
mike@ME-Ubuntu:~$ for (( i = 0; i < 2; i++ ))
> do
> echo "number is $i"
> done
number is 0
number is 1
```

Loops: while

- Form:

  while [conditional expression]

  do

  statements

  done

Loops: while

e.g.:

```
>> count=0
>> while [ $count -lt 2 ]
> do
>echo “The number is $((count++))”
> done
The number is 0
The number is 1
```

Loops: while

```
mike@ME-Ubuntu:~$ count=0
mike@ME-Ubuntu:~$ while [ $count -lt 2 ]
> do
> echo "The number is $((count++))"
> done
The number is 0
The number is 1
```

Functions

Functions:

- Form:

function function_handle () {

statements

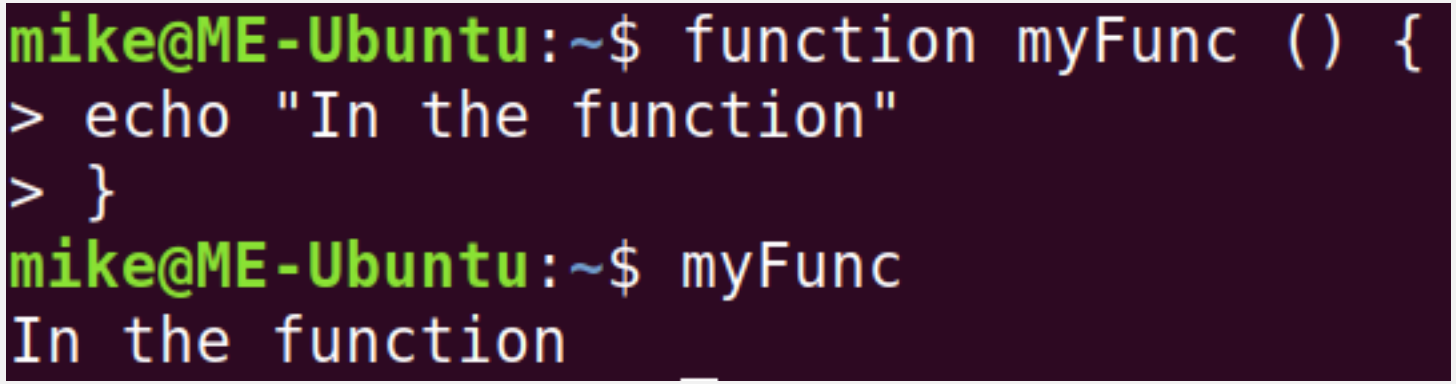
}

This will define the function called *function_handle* in the current namespace, for use later.

Functions: Calling

e.g.:

```
>> function myFunc () {
> echo “In the function”
> }
>> myFunc
In the function
```



Functions: Input Arguments

- Arguments are passed in by a list of variables named as sequential numbers
- **$1, $2, ... , $n** where there are n arguments
- **$0** is reserved and denotes the function handle
- Not sure how many were passed in?
  - Use **$#**

Functions: Input Arguments

e.g.:

```
>> function myFunc () {
>   echo $#
> echo $2
> }
>> myFunc cat dog
2
dog
```

```
mike@ME-Ubuntu:~$ myFunc cat dog
2
dog
```

Functions: Returning Values

- Values are returned with **echo** putting a value to **stdout**
- From stdout we can then **redirect** and **pipe** or assign into variables etc.
- The **return** keyword is different, this terminates the function and provides a status code

Functions: Returning values

e.g.:

```
>> function gimme_five () {
>  echo 5
> }
>> myVar=$(gimme_five)
>> echo $myVar
5
```

```
mike@ME-Ubuntu:~$ function gimme_five () { echo 5; }
mike@ME-Ubuntu:~$ myVar=$(gimme_five)
mike@ME-Ubuntu:~$ echo $myVar
 5
```

Scripts

Scripts: Writing

- We want to create a script of potentially many command calls and conditional flow
- Create .sh extension file containing BASH syntax
- We can use a variety of methods to write a shell script:
  - Editors like nano, vim, notepad++..
  - Commands like echo redirected to a file ...

Scripts: Writing a file with the nano application

- Nano application will allow you to edit a file.
- Can move back and forth through the lines and edit.
- Saving to file is then *<CTRL>+O* to write out
- Enter filename and press <ENTER>
- *<CTRL>+X* to exit nano

Executing Scripts

- Running the script requires executing it via invoking the shell on the script:
  - sh scriptname.sh
  - bash scriptname.sh

e.g.:

```
>>  sh  return_example.sh # from previous slide
some result
```

Executing Scripts: The Shebang/Hashbang

- Depending on the current shell, we may need to point to the correct interpreter
- A Shebang / Hashbang is a path to the interpreter
- Sits at the top of the script
- Can also help an IDE to highlight syntax

Executing Scripts: The Shebang/Hashbang

- #!/bin/bash

This points us to the BASH interpreter and allows us to use BASH commands where needed.

return_example.sh

```
#!/bin/bash

my_function () {
  local func_result="some result"
  echo "$func_result"
}

func_result="$(my_function)"
echo $func_result
```

Wild Cards

Wild Cards

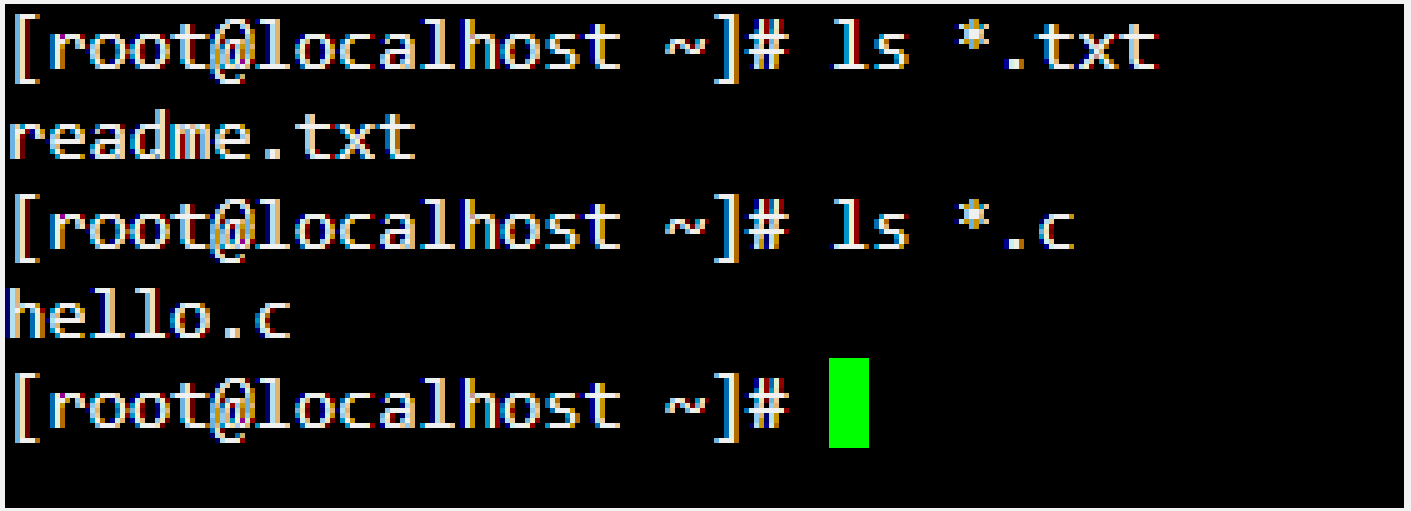
- *
- ?
- []
- {}

*

- Match all characters.

e.g.

```
ls *.txt
ls *.c
```



?

- Match for a single character.

e.g.

```
cat *.?
cat *.??
```

```
[root@localhost ~]# cat *.?
/* This C source can be compiled with:
   gcc -o hello hello.c
*/
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("Hello World\n");
    return 0;
[root@localhost ~]# cat *.??
def fib(n):
    if n < 2:
        return n
    return fib(n-2) + fib(n-1)

n = 25
print "fib(", n, ")= ", fib(n)
[root@localhost ~]#
```

[ ]

- Match range of values.

e.g.

```
touch test1.txt
touch test2.txt
touch test3.txt
ls test[1-2].txt
rm test1.txt
ls
```

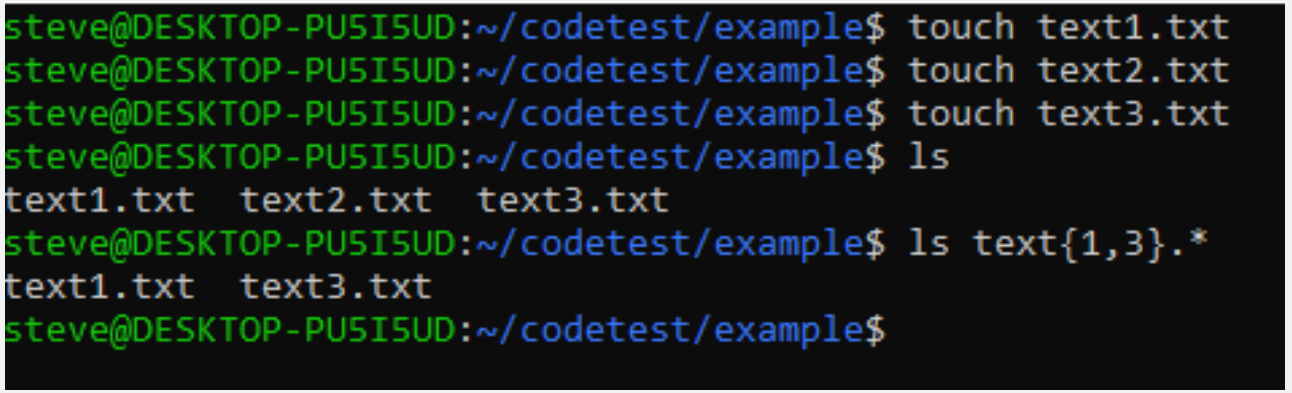
```
[root@localhost ~]# touch test1.txt
[root@localhost ~]# touch test2.txt
[root@localhost ~]# touch test3.txt
[root@localhost ~]# ls
bench.py       long           rv128test.bin  test2.txt      testFolder
hello.c        readme.txt     test1.txt      test3.txt
[root@localhost ~]# ls test[1-2].*
test1.txt  test2.txt
[root@localhost ~]# rm test1.txt
[root@localhost ~]# ls
bench.py       long           rv128test.bin  test3.txt
hello.c        readme.txt     test2.txt      testFolder
[root@localhost ~]#
```

{ }

- Match list of values.

e.g.

```
touch test1.txt
touch test2.txt
touch test3.txt
ls
ls text{1,3}.txt
```



Process Control

- ***<CTRL>+c***
- ***<CTRL>+z***
- ***<CTRL>+a***
- ***<CTRL>+e***
- ***<CTRL>+k***
- ***<CTRL>+u***
- ***<CTRL>+y***
- ***<CTRL>+d***

<CTRL>+C and <CTRL>+Z

- *<CTRL>+C*
- will attempt to kill whatever process you are currently running
- *<CTRL>+Z*
- will put the running process into a suspended background process.
- the *fg* command restores it, moving it back to the foreground.

<CTRL>+A and <CTRL>+E

- *<CTRL>+A*
- moves the cursor to the start of the line you are currently on.
- *<CTRL>+E*
- moves the cursor to the end of the line you are currently on.

<CTRL>+K, <CTRL>+U, and <CTRL>+Y

- *<CTRL>+U*
- clear the line **before** the cursor and paste into the buffer.
- *<CTRL>+K*
- clears the line **after** the cursor and pastes it into the buffer.
- *<CTRL>+Y*
- pastes the buffer at the current cursor location.

<CTRL>+D

- *<CTRL>+D*
- exits the current shell

Goals for Today

- Variables and arrays in bash: assignment, quoting and indexing
- Functions: defining them and returning values
- Writing and running scripts: nano, the shebang and file permissions
- Wildcards: matching filenames with * and ?
- Process control and editing keys: CTRL+C, CTRL+Z, fg, CTRL+K/U/Y

CS-107 Lecture 21
Scripting, Permission, Wildcards and Process Control

Troy Astarte

Megan Venn-Wycherley